

Breaking Barriers: Expanding GPU Memory with Sub-Two Digit Nanosecond Latency CXL Controller

Donghyun Gouk*, Seungkwan Kang†, Hanyeoreum Bae†, Eojin Ryu*, Sangwon Lee*, Dongpyung Kim*, Junhyeok Jang†, Myoungsoo Jung*†

†Computer Architecture and Memory Systems Laboratory, KAIST

*Pannnesia, Inc.

<https://pannnesia.com>

Abstract

This work introduces a GPU storage expansion solution utilizing CXL, featuring a novel GPU system design with multiple CXL root ports for integrating diverse storage media (DRAMs and/or SSDs). We developed and siliconized a custom CXL controller integrated at the hardware RTL level, achieving two-digit nanosecond roundtrip latency, the first in the field. This study also includes speculative read and deterministic store mechanisms to efficiently manage read and write operations to hide the endpoint’s backend media latency variation. Performance evaluations reveal our approach significantly outperforms existing methods, marking a substantial advancement in GPU storage technology.

CCS Concepts: • **Hardware** → **Emerging interfaces; Memory and dense storage.**

Keywords: CXL, Memory Expansion, GPGPU

ACM Reference Format:

Donghyun Gouk*, Seungkwan Kang†, Hanyeoreum Bae†, Eojin Ryu*, Sangwon Lee*, Dongpyung Kim*, Junhyeok Jang†, Myoungsoo Jung*†. 2024. Breaking Barriers: Expanding GPU Memory with Sub-Two Digit Nanosecond Latency CXL Controller. In *16th ACM Workshop on Hot Topics in Storage and File Systems (HOTSTORAGE ’24)*, July 8–9, 2024, Santa Clara, CA, USA. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/3655038.3665953>

1 Introduction

Large-scale deep learning models, including large language models and mixtures of experts, have emerged across various computing domains. A primary challenge with these models is that their memory footprints and requirements significantly

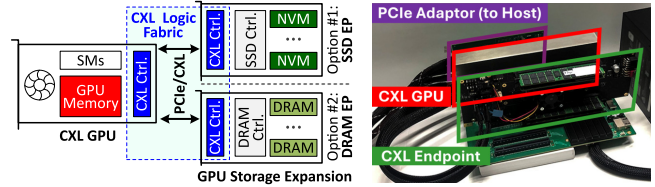
Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

HOTSTORAGE ’24, July 8–9, 2024, Santa Clara, CA, USA

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-0630-1/24/07.

<https://doi.org/10.1145/3655038.3665953>



(a) Overview.

(b) AIC-level prototype.

Fig. 1. A high-level view of GPU storage expansion.

exceed the available memory capacity of accelerators, particularly GPUs. For instance, while loading and training models with 1 billion parameters necessitate 16~24GB of GPU memory, systems utilizing models with over 100 billion parameters are increasingly common [1–6].

To address these challenges, various strategies have been devised that utilize storage solutions [7–10] or external memory [11–14]. One such solution is GPU direct storage (GPUDirect [15]), which allows for direct mapping of the GPU’s PCIe base address register (BAR) to a target SSD by modifying the storage stack. This method enables the management of large-scale model parameters by leveraging the extensive storage capacity of SSDs. Despite its potential, GPUDirect is infrequently used due to its complexity. It treats the SSD as a block device, requiring meticulous management of the file system and the inherent disparity in I/O granularity between memory and block accesses. Moreover, GPUDirect necessitates manual handling of storage/memory operations (e.g., `cuFileWrite`), complicating the copy-then-execute programming model and hindering its adoption.

In contrast, *unified virtual memory* (UVM [16]) offers a simpler solution by allowing both CPU and GPU to access data through a shared virtual memory space that includes both GPU and host memories. This system allocates and migrates memory automatically on demand, making UVM a more commonly used approach for managing large datasets in deep learning applications such as TensorFlow [17] and DGL [18].

In this study, we introduce the *compute express link* (CXL) as a valuable technology for expanding GPU storage. Refer to Figure 1a for a high-level overview of the GPU storage expansion approach this paper aims to facilitate. CXL employs the PCIe physical data link to map devices, known as *endpoints* (EPs), to a cacheable memory space accessible by the host. This setup allows compute units to access EPs directly through standard memory requests. Unlike the

synchronous communication of DRAM interfaces defined by JEDEC’s DDR [19] standard, CXL enables asynchronous communication with compute units, similar to block storage. This flexibility permits the implementation of EPs using various storage media, including non-volatile memory (NVM) based SSDs [20–22] and DRAMs [23–28].

Nonetheless, integrating CXL for GPU storage expansion poses a notable challenge due to the current lack of a CXL logic fabric and subsystem in GPUs that can support DRAM/SSD EPs as memory expansion devices. To overcome this, we have developed a CXL hardware layer stack and realized it through the design of a CXL controller integrated at the hardware RTL within any component of the CXL logic fabric. Following this, we propose a GPU architecture featuring multiple CXL root ports equipped with these CXL controllers, each capable of supporting DRAM/SSD EPs via a host bridge. This effort marks the first demonstration of a real, silicon-based CXL controller achieving two-digit nanosecond round-trip latency, showcasing significant advancements in high-speed memory expansion technology.

We further refined the design of our CXL controller to improve GPU storage expansion capabilities. Despite achieving the fastest CXL round-trip latency to date, the storage media of EPs remains slower compared to local memory. To address this discrepancy, we introduce two strategies from the perspective of GPU interaction with EPs: i) *speculative read* (SR) and *deterministic store* (DS). Our controller features an additional module designed to manage load instructions with SR, a feature from CXL 2.0, which anticipates target addresses by pre-sharing them with the DRAM/SSD EPs. SR enables EPs to prefetch target pages before receiving the actual request, significantly reducing the latency impact of their storage media. However, considering the potential load SR requests might place on EPs, we monitor EP status via the CXL quality of service (QoS) telemetry feature. This allows for precise management of SR request traffic, balancing performance and system load.

Conversely, in situations where storage media exhibit varied write performance, addressing the differences in write speeds is crucial. The DS mechanism is tailored for scenarios involving slower EP media, such as NVM-based SSDs. It executes concurrent writes to both the GPU memory and the SSD EP, immediately completing the store request from compute resources. Given that GPU operations are predominantly read-oriented, this “fire-and-forget” strategy is highly effective for most real-world applications. Nonetheless, in cases marked by intensive write activities or significant SSD write latencies (a scenario referred to as a tail case), the DS mechanism identifies the SSD EP as a bottleneck. Our controller then temporarily stores upcoming data in GPU memory, subsequently transferring them to the SSD EP. This approach ensures that the GPU can maintain the fire-and-forget method for the majority of store operations, significantly boosting performance across a wide range of use cases.

For the performance evaluation of our CXL hardware framework, we fabricated the CXL controller using cutting-edge silicon technology. We achieved integration with the GPU by utilizing Vortex [29], a RISC-V based general-purpose GPU. Our controller’s RTL design was embedded into this environment. The effectiveness of our approach to GPU storage expansion was rigorously tested with our custom-designed add-in-card (AIC) devices, as illustrated in Figure 1b. We carried out comprehensive tests using various storage media and GPU storage expansion techniques via RTL-level simulation of hardware. Results from these tests indicate that our GPU storage expansion method significantly outperforms both the UVM strategy and a commercial EP prototype controller [25] adopted for GPU storage integration, showing $2.36\times$ and $1.36\times$ higher performance, respectively.

The main contributions of this paper can be summarized as follows:

- *Design of CXL-integrated GPU*: We design a GPU’s CXL root port and GPU internal architecture to enable the GPU device to directly access a memory expander without a host intervention.
- *Demonstration of real, silicon-based CXL controller*: To achieve high-speed memory expansion, we show CXL silicon stack of a low-latency CXL controller and integrate it to GPU hardware at the hardware RTL level.
- *Speculative read and deterministic store mechanisms*: While speculative read anticipates target addresses and enables endpoint to prefetch target pages, deterministic store manages concurrent writes to GPU memory and SSD EPs.

2 Data Movement and CXL

2.1 Data Movement in GPUs

Copy-then-execute model. GPU memory plays a critical role not only in storing model parameters but also in accommodating other data types such as training metadata, gradients, and intermediate computation buffers [9, 11, 30–32]. The memory demands of large-scale deep learning models are typically around eight times greater than the memory needed for storing their parameters alone [9, 11, 30, 32], often surpassing the capacity of current GPUs, which is limited to 80 GB [33]. To address this limitation, several strategies, including various models of parallelism that utilize the resources of multiple GPUs, have been proposed [34–37]. While these approaches can reduce the data load on individual GPUs, fully accommodating all necessary data within GPU memory remains impractical. As a result, GPUs often have to rely on host memory and storage, requiring frequent data transfers between the CPU and GPU. Figure 2 depicts the GPU’s “copy-then-execute” programming model for large-scale training, which involves dividing model parameters into segments, or tiles, each associated with a different layer of the training model. This segmentation facilitates the swapping of layer output vectors (①), thus allowing subsequent layers to process

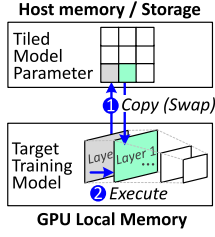


Fig. 2. Data movement.

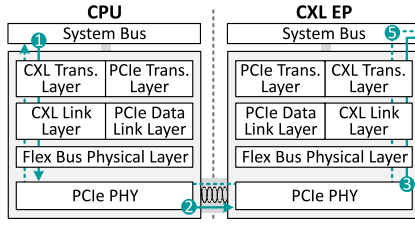


Fig. 3. Latency analysis.

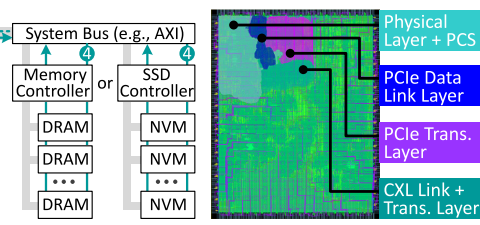


Fig. 4. Silicon layout.

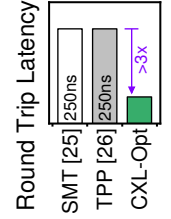


Fig. 5. Latency.

using their designated model parameters (2). However, the requisite data movements in this model not only introduce significant performance overheads but also pose a considerable barrier to GPU adoption [8, 14]. This complexity arises from the need for users to explicitly manage data structures, oversee the transfer of tiled data, and coordinate data migration, processes that can be both cumbersome and inefficient.

Unified virtual memory. NVIDIA and AMD support their GPU architectures with runtime software for UVM [16, 38]. UVM allows both the CPU and GPU to access data through a shared pointer within a unified virtual address space. Should a memory access request occur for data not currently in the GPU’s memory, this triggers a cache miss and a subsequent page fault. This event prompts a PCIe interrupt, signaling to the host. The host-side runtime software responds by allocating a new page in the GPU memory, transferring the needed data to this page, and updating the GPU. Essentially, memory in the virtual space is allocated on-demand. Furthermore, to maintain adherence to a predetermined memory budget, UVM seamlessly replaces older pages in the GPU memory with new data migrations. Thanks to these capabilities, UVM has found widespread use in various frameworks, including TensorFlow [17], DGL [18], and OneAPI [39].

Despite the advantages of UVM in enhancing GPU accessibility, it may induce performance bottlenecks. This performance drop is largely due to the significant latency incurred when the host runtime intervenes to address GPU page faults.

2.2 CXL Memory Expander

CXL memory interface. The CXL is an open-industry protocol designed to integrate the memory space exposed by underlying endpoint (EP) devices with cacheable system memory [40]. It incorporates three sub-protocols: *CXL.cache*, *CXL.io*, and *CXL.mem*. *CXL.cache* is focused on managing cache coherency across various computing resources, ensuring that data accessed by different processors remains consistent. *CXL.mem* offers a read and write interface through PCIe, handling all communications via packets, or CXL flits, facilitating asynchronous memory accesses that move beyond the traditional limitations set by JEDEC memory interfaces. This adaptability allows it to support a range of media types, including NVM SSD and various versions of DDR DRAM. *CXL.io* functions similarly to PCIe, mainly facilitating device enumeration and bulk I/O communications. Among the protocol’s noteworthy applications is the CXL memory expander,

which stands out in industry discussions for its potential to significantly improve computing infrastructure [24, 26].

Detailed end-to-end latency analysis. Many industry vendors, including Samsung, Micron, and SK-Hynix, have developed prototypes or proofs-of-concept for CXL memory expanders that leverage CXL 2.0 [41–43]. However, as of the present, commercially available products or in-depth latency information from an end-user perspective remains scarce. This subsection aims to detail the components contributing to CXL’s end-to-end latency.

CXL’s latency is distinctive from the JEDEC/DDR standards, which are characterized by precise timing specifications. Instead, CXL latency encompasses the full round-trip path across all relevant layers. Figure 3 provides an in-depth look at the hardware layer stack, extending from the host to an EP. A memory request from the host CPU is converted into a CXL flit at the transaction layer, which supports the functionalities and interfaces required by each sub-protocol. This flit progresses to the link layer, which oversees the flow of communications, including buffering and acknowledgments. The flit is then passed to the Flex Bus physical layer (1), which sends it to the target EP based on factors such as link speed and lane configuration (2). For accurate management of CXL subprotocols, the EP must integrate these layers as well. After navigating the EP’s layer stack, the flit returns to the system bus (3), reaching either a memory controller or an SSD controller. The data is then retrieved from the storage medium (4) and sent back to the host CPU’s system bus, traversing the hardware layers once again (5). The PCIe transaction and link layers are crucial components due to their roles in device enumeration, configuration, and data management.

The structure and function of this hardware layer stack play a critical role in the determination of round-trip latency. Although market products are yet to be released, studies conducted by organizations like Samsung [25] and Meta [26] have examined their prototypes, reporting a latency of 250ns.

3 GPU Storage Expansion over CXL

3.1 Designing CXL-Integrated GPU

CXL hardware layer stack. In response to the lack of a publicly accessible, comprehensive hardware stack for CXL, we have developed a series of essential hardware layers that support the three key CXL subprotocols, consolidating these into a unified controller. Figure 4 illustrates the silicon layout of our CXL controller, fabricated using advanced silicon

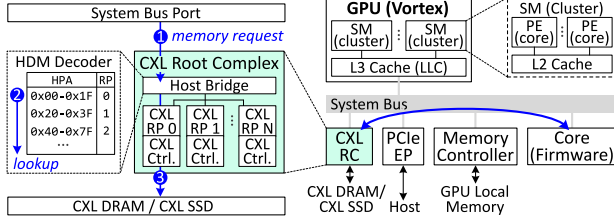


Fig. 6. GPU architecture.

technology. This controller is designed for compatibility with CXL 3.1 while also offering backward compatibility with earlier versions of CXL (2.0/1.1). The Flex Bus physical layer is integrated with our PCIe physical coding sublayer (PCS), allowing it to support both PCIe and CXL layer stacks seamlessly over elastic buffers [40, 44].

To effectively handle the dual demands of PCIe and CXL, particularly in terms of power management and administrative tasks, an arbitrator state machine has been implemented within the controller. This state machine ensures efficient allocation of resources between PCIe and CXL operations, maintaining optimal performance and stability. The controller has been rigorously tested, demonstrating a two-digit nanosecond round-trip latency, which includes the time taken for protocol conversion between standard memory operations and CXL flit transmissions. It has been successfully integrated into both a memory expander and GPU/CPU prototypes at the hardware RTL, showcasing its effectiveness and compatibility across a range of computing hardware.

Please note that, for better understanding, we compare the round-trip latency of our CXL controller with that of SMT [25] and TPP [26]. The results are shown in Figure 5, demonstrating that our controller is more than $3\times$ faster than SMT and TPP. Although there is no publicly available information about their silicon, we conjecture that they leverage PCIe architecture. In contrast, our controller is fully optimized for CXL, from the physical layer to the link layer, and the transaction layer.

GPU architecture design and integration. In our approach to developing EP devices for GPU storage expansion, we have combined the functionalities of the CXL controller with those of memory/SSD controllers, as shown in the round-trip datapath shown in Figure 3. This combination enables the CXL controller to extend its backend storage space to the host system, designated as *host-managed device memory* (HDM). This integration is communicated to the host through a feature similar to PCIe capabilities, tailored for HDM usage.

However, integrating with the GPU architecture presents unique challenges due to the requirement for the EP to be recognized by the GPU’s cache system. A straightforward implementation, akin to that used for EPs, is not suitable in this context. To address these challenges, we have developed a specialized CXL root complex featuring a host bridge equipped with multiple root ports. This configuration is shown in Figure 6, where the host bridge connects to a system bus port on one

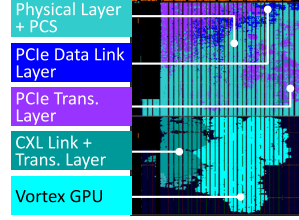


Fig. 7. RTL floorplan.

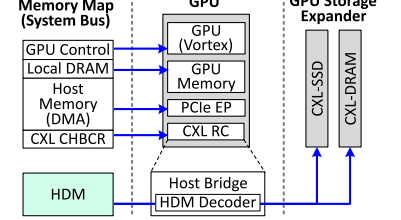


Fig. 8. GPU memory space.

side and several CXL root ports on the other. One of the key components of this setup is an HDM decoder, responsible for managing the address ranges of system memory, referred to as *host physical address* (HPA), for each root port. These root ports are designed to be flexible, capable of supporting either DRAM or SSD EPs via PCIe connections. When a GPU compute unit issues a memory request (1), the designated root port interprets this request into a CXL flit format (2) and forwards it to the CXL controller (3). This process facilitates seamless communication between the GPU and the expanded storage, ensuring efficient data handling and accessibility.

System configuration and memory space. Figure 6 shows the placement of the CXL root complex within our GPU architecture, which utilizes the Vortex GPU framework and the corresponding hardware RTL [29]. In Vortex, the GPU’s computational units are termed streaming multiprocessors (SMs) or clusters, all of which connect to the system bus via the last-level cache (LLC). In addition, the system bus facilitates connections to the controller for GPU local memory and to a PCIe EP, serving both host communication needs and kernel execution. Our design incorporates the CXL root complex into the system bus, complemented by a simplified core. This core is responsible for initializing EPs connected to the root complex, the HDM decoder of host bridge, and the associated HPAs ranges. Firmware identifies CXL EPs by inspecting their configuration space and the PCIe BAR, then aggregates each EP’s memory address space by examining its HDM capability register. This information is subsequently recorded in the host bridge’s HDM decoder, noting the HDM’s base address and size for each root port (Figure 6).

Following this initialization phase, the GPU’s system bus memory map is detailed in Figure 8. This map delineates the allocation of address space across different devices, segmented according to their specific functions. For instance, when an SM issues a request targeting host memory, it engages the PCIe EP to route the request to the host. Consequently, we integrate the GPU storage expanders, managed by the root complex, into the system’s memory architecture. Upon receiving a request for this segment of system memory, the CXL root complex reviews the HDM decoder, converts the memory request into a CXL flit, and dispatches it to the appropriate CXL root port and controller. The systems that integrate our CXL controllers, including the Vortex GPU and EP, are realized on an 7 nm FPGA-based custom AIC, as depicted in Figure 1b, and their RTL are shown in Figure 7.

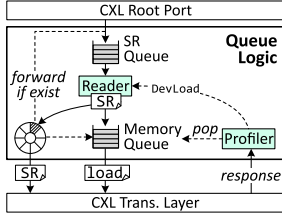


Fig. 9. Speculative read.

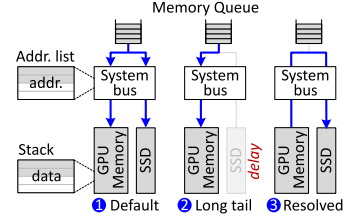


Fig. 10. Deterministic store.

3.2 Tailoring CXL Controller for GPUs

Our CXL controller attains a round-trip latency in the two-digit range; however, this latency is subject to variation depending on the type of media access, such as DRAM or SSD. To mitigate the impact of backend media latency, we propose two optional strategies: i) speculative read (SR) and ii) deterministic store (DS). The SR strategy is aimed at reducing read latency, whereas the DS approach focuses on addressing the slower or tail latencies associated with write operations.

Speculative read. We utilize the speculative read feature (MemSpecRd) introduced in CXL 2.0 to enhance our system’s efficiency. MemSpecRd functions similarly to standard read or write requests but specifies an address for a page likely to be accessed shortly. As shown in Figure 9, beneath the root port, our implementation includes queue logic with two separate queues: the SR queue and the memory queue, each capable of holding 32 entries. An incoming load request is placed into the SR queue, where its reader module then generates and dispatches an SR request. At this juncture, we adapt the MemSpecRd address format for our specific use: traditionally intended for 64B granularity, we adjust the two least significant bits to represent the request’s length, while the remaining bits signal a 256B granular memory offset. This modification enables the amalgamation of multiple memory request details (1~4) into a singular MemSpecRd request.

Should the memory queue have available space, the SR-generated request is transferred there; if not, it remains pending in the SR queue. Requests within the memory queue are then conveyed to the CXL controller’s transaction layer, ensuring the target EP can process them. In addition, the SR reader module records each issued request’s address and length within a ring buffer. Consequently, if a new request’s address matches any previously issued SR request, it is directly forwarded as a memory request. By preemptively transmitting most load addresses to the EP, the EP-side CXL controller can prefetch the data before the actual read requests.

Upon receiving a response from the target EP, the queue logic’s profiler removes the relevant request from the memory queue. Note that SR requests can also contribute PCIe traffic. To address this, the profiler also assesses the status of the underlying EP by analyzing the QoS telemetry data from the response, particularly the DevLoad field. The DevLoad metric is shared with the SR reader to modulate the volume of SR requests accordingly; a higher DevLoad value indicates busier device, prompting a reduction in SR request frequency.

Deterministic store. Several industry proposals have emerged suggesting the integration of SSDs with CXL. Notably, Intel has announced the release of a CXL-attached Optane SSD [45], and Samsung has introduced a CXL hybrid SSD (CMM-H [46]). Although actual products based on these announcements are yet to materialize, it is anticipated that write operations on these devices may exhibit slower speeds compared to read operations, or they may occasionally encounter tail latency due to internal activities of the SSD controller. The DS technique offers a solution to conceal such characteristics by leveraging reserved space in GPU memory. As shown in Figure 10, when a write operation to an SSD is initiated in the memory queue, the request is concurrently sent to both the GPU memory and the SSD (as orchestrated by the system memory map), allowing for the immediate release of the request (❶). Should there be a delay observed from the SSD prior to the arrival of the subsequent write request, the DS logic temporarily stores the data in the GPU memory’s reserved address (❷). This temporary data storage is organized using a stack structure, designed to collapse in the event of detected tail latency or slow writes. Conversely, if no such issues are observed, the data within the GPU memory at the designated address is updated, reflecting the dual write operation (GPU memory/SSD). To maintain a record of each stack entry’s precise location, an address list is stored within the system bus’s internal SRAM. The stack is methodically emptied by flushing the data in the background (❸).

Through the implementation of DS, from the perspective of the SMs and their LLC controller, the CXL root complex exhibits deterministic store characteristics, effectively shielding them from variations in write latency.

4 Evaluation

Methodology. While our hardware prototype can provide detailed latency characteristics that are unavailable for end users, it lacks the flexibility needed to explore design spaces of GPU storage expansion. Therefore, we established a simulator that accurately models our prototype and used it for the evaluation. The simulator replicates our RTL logic’s behavior by using the information extracted from the real-world execution of workloads and RTL behavior simulation. For example, we used the execution time and GPU pipeline statistics extracted from Vortex’s performance counter and waveform dump. Memory latencies of Vortex GPU and CXL endpoints are calculated using DRAMSim3 [47]. For the PCIe/CXL bus latencies, we used real values measured from our ASIC. Further details of our simulation setup are in Table 1.

Configurations. We compare four different GPU systems addressing large data challenge by implementing them on our simulator. Specifically, UVM provides a virtual address space shared across the host and GPU by on-demand memory allocation via page fault. CXL-Proto expands the GPU storage by employing a CXL endpoint prototype that exhibits

Vortex Configuration	Workload	Cycles	Store Ratio
Cores / Clusters 4 / 2			
PCIe Configuration	BFS	35K	46%
PCIe 5.0 (32GT/s) x8	SpMV	32K	9%
CXL Configuration	GEMM	226K	5%
PCIe 5.0 (32GT/s) x8	Conv	2M	97%
Sync-header bypass	VecAdd	29K	94%

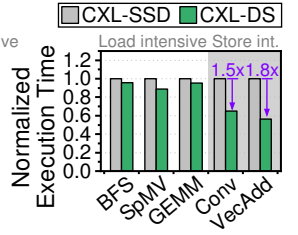
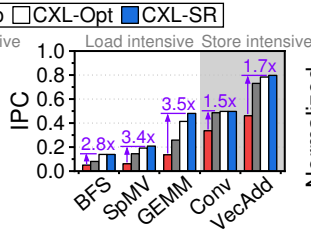
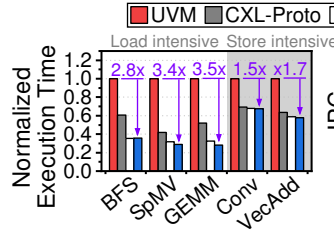


Table 1. Setup. **Table 2.** Workloads. **Fig. 11.** Kernel execution time. **Fig. 12.** IPC. **Fig. 13.** Deterministic Store.

250 ns latency, following Samsung and Meta’s report [25, 26]. CXL-Opt also implements GPU storage expansion via CXL, but employs our CXL controller with two-digit nanosecond round-trip latency. CXL-SR applies our speculative read (SR) strategy on CXL-Opt to further shorten the read latency.

In addition, we compare two more CXL-based systems to evaluate our deterministic store strategy: CXL-SSD employs Intel’s CXL-attached Optane SSD [45], while CXL-DS applies deterministic store (DS) strategy atop CXL-SSD.

Workloads. We used five different GPU kernels that are widely used for evaluating GPU systems: Breadth-First Search (BFS [48]), Convolution (Conv), GEMM [49], Vector Add (VecAdd), SpMV [49]. For reference, we measured important characteristics of the workloads using UVM and summarized them in Table 2.

Performance Analysis. Figure 11 compares the execution time of four different systems, and the values are normalized to that of UVM. The workloads are sorted by the IPC measured using UVM. As shown in the figure, UVM exhibited the worst performance across all GPU kernels we tested. This is because UVM suffers from host runtime intervention overhead when handling GPU page faults. In addition, since UVM migrates the data in the granularity of a page rather than a cacheline, the amount of data transfer is often larger than what the GPU kernel actually needs. This problem amplifies when the GPU kernel mostly shows a random access pattern, as in BFS, causing a significant slowdown. In contrast, CXL allows GPU to access the expanded storage via load/store instructions, thereby eliminating the host runtime intervention and unnecessary data transfer issues. This makes CXL-Proto’s execution time 1.8× shorter than UVM.

Our CXL-Opt further shortens the execution time by 1.34× by employing an optimized CXL controller exhibiting two-digit nanosecond round-trip latency. However, CXL-Opt shows less performance gain for Conv and VecAdd compared to other workloads. This is because these workloads suffer less from memory stalls. This can also be observed in Figure 12, which shows the IPC values collected during GPU kernel execution. We can see that the three workloads, suffering less memory stall, show higher IPC than others on CXL-Proto. Note that BFS, whose IPC is the lowest, wastes 40% of its execution time idling due to the memory stall. Lastly, CXL-SR shows 1.08× faster performance than CXL-Opt. This is because CXL-SR issues SR requests ahead of actual memory

requests, hiding the memory latency behind computing time as discussed in section 3.2.

Deterministic Store. Figure 13 compares the kernel execution time of CXL-SSD and CXL-DS. For clear understanding, the values are normalized to the execution time of CXL-SSD. We can see that CXL-DS shows a significant performance gain of 1.65× for VecAdd and Conv, whose memory accesses are mostly writes ($\geq 93\%$). This is because CXL-DS can alleviate the slowdown due to the long write latency of OptaneSSD. This is done by leveraging GPU memory as a temporary data store to manage the write operations with the fire-and-forget method as described in section 3.2. On the other hand, CXL-DS showed relatively less performance gain of 7.36% for the remaining workloads. This is because the remaining workloads issue fewer write requests ($\leq 46\%$), making the performance less sensitive to write latencies.

Note that our evaluations were done using Intel OptaneSSD, which hardly suffers from long-tail latency due to internal activities. We believe CXL-DS’s performance improvements would be more significant if we use NAND flash (e.g., Samsung Z-NAND [50]) as the endpoint’s storage media.

5 Conclusion

We introduce a method to increase GPU storage using CXL technology, integrating DRAM and/or SSDs through CXL multiple root ports. Our custom-designed CXL controller, implemented directly in the hardware, achieves fast response times not seen before in this field. We further refine this system with features designed to improve handling of both read and write operations to storage media. Our tests show that this approach improves upon existing solutions, offering a significant step forward in GPU storage capacity and efficiency.

6 Acknowledgments

The authors thank the anonymous referees for their helpful comments, which improved the quality of the manuscript. The authors also thank Javier Gonzalez for shepherding this paper. Authors S. Kang, H. Bae, and J. Jang associated with KAIST contributed to introducing the concept of speculative read and deterministic store, with all remaining work and contributions brought by Panmnesia. This work is protected by one or more patents.

References

- [1] Dheevatsa Mudigere, Yuchen Hao, Jianyu Huang, Zhihao Jia, Andrew Tulloch, Srinivas Sridharan, Xing Liu, Mustafa Ozdal, Jade Nie, Jongsoo Park, et al. Software-hardware co-design for fast and scalable training of deep learning recommendation models. In *Proceedings of the 49th Annual International Symposium on Computer Architecture*, pages 993–1011, 2022.
- [2] An Yang, Junyang Lin, Rui Men, Chang Zhou, Le Jiang, Xianyan Jia, Ang Wang, Jie Zhang, Jiamang Wang, Yong Li, et al. M6-t: Exploring sparse expert models and beyond. *arXiv preprint arXiv:2105.15082*, 2021.
- [3] Ross Taylor, Marcin Kardas, Guillem Cucurull, Thomas Scialom, Anthony Hartshorn, Elvis Saravia, Andrew Poulton, Viktor Kerkez, and Robert Stojnic. Galactica: A large language model for science. *arXiv preprint arXiv:2211.09085*, 2022.
- [4] Romal Thoppilan, Daniel De Freitas, Jamie Hall, Noam Shazeer, Apoorv Kulshreshtha, Heng-Tze Cheng, Alicia Jin, Taylor Bos, Leslie Baker, Yu Du, et al. Lamda: Language models for dialog applications. *arXiv preprint arXiv:2201.08239*, 2022.
- [5] Rohan Anil, Andrew M Dai, Orhan Firat, Melvin Johnson, Dmitry Lepikhin, Alexandre Passos, Siamak Shakeri, Emanuel Taropa, Paige Bailey, Zhifeng Chen, et al. Palm 2 technical report. *arXiv preprint arXiv:2305.10403*, 2023.
- [6] Danny Driess, Fei Xia, Mehdi SM Sajjadi, Corey Lynch, Aakanksha Chowdhery, Brian Ichter, Ayzaan Wahid, Jonathan Tompson, Quan Vuong, Tianhe Yu, et al. Palm-e: An embodied multimodal language model. *arXiv preprint arXiv:2303.03378*, 2023.
- [7] Weijie Zhao, Jingyuan Zhang, Deping Xie, Yulei Qian, Ronglai Jia, and Ping Li. Aibox: Ctr prediction model training on a single node. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management*, pages 319–328, 2019.
- [8] Jonghyun Bae, Jongsung Lee, Yunho Jin, Sam Son, Shine Kim, Hakbeom Jang, Tae Jun Ham, and Jae W Lee. {FlashNeuron}:{SSD-Enabled}{Large-Batch} training of very deep neural networks. In *19th USENIX Conference on File and Storage Technologies (FAST 21)*, pages 387–401, 2021.
- [9] Samyam Rajbhandari, Olatunji Ruwase, Jeff Rasley, Shaden Smith, and Yuxiong He. Zero-infinity: Breaking the gpu memory wall for extreme scale deep learning. In *Proceedings of the international conference for high performance computing, networking, storage and analysis*, pages 1–14, 2021.
- [10] Weijie Zhao, Deping Xie, Ronglai Jia, Yulei Qian, Ruiquan Ding, Mingming Sun, and Ping Li. Distributed hierarchical gpu parameter server for massive scale deep learning ads systems. *Proceedings of Machine Learning and Systems*, 2:412–428, 2020.
- [11] Minsoo Rhu, Natalia Gimelshein, Jason Clemons, Arslan Zulfiqar, and Stephen W Keckler. vdn: Virtualized deep neural networks for scalable, memory-efficient neural network design. In *2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 1–13. IEEE, 2016.
- [12] Xiaoming Chen, Danny Z Chen, and Xiaobo Sharon Hu. modnn: Memory optimal dnn training on gpus. In *2018 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, pages 13–18. IEEE, 2018.
- [13] Linnan Wang, Jinmian Ye, Yiyang Zhao, Wei Wu, Ang Li, Shuaiwen Leon Song, Zenglin Xu, and Tim Kraska. Superneurons: Dynamic gpu memory management for training deep neural networks. In *Proceedings of the 23rd ACM SIGPLAN symposium on principles and practice of parallel programming*, pages 41–53, 2018.
- [14] Chien-Chin Huang, Gu Jin, and Jinyang Li. Swapadvisor: Pushing deep learning beyond the gpu memory limit via smart swapping. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 1341–1355, 2020.
- [15] NVIDIA. Nvidia gpudirect.
- [16] NVIDIA. Cuda toolkit documentation.
- [17] Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Greg S. Corrado, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Ian Goodfellow, Andrew Harp, Geoffrey Irving, Michael Isard, Yangqing Jia, Rafal Jozefowicz, Lukasz Kaiser, Manjunath Kudlur, Josh Levenberg, Dandelion Mané, Rajat Monga, Sherry Moore, Derek Murray, Chris Olah, Mike Schuster, Jonathon Shlens, Benoit Steiner, Ilya Sutskever, Kunal Talwar, Paul Tucker, Vincent Vanhoucke, Vijay Vasudevan, Fernanda Viégas, Oriol Vinyals, Pete Warden, Martin Wattenberg, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. TensorFlow: Large-scale machine learning on heterogeneous systems, 2015. Software available from tensorflow.org.
- [18] Minjie Wang, Da Zheng, Zihao Ye, Quan Gan, Mufei Li, Xiang Song, Jinjing Zhou, Chao Ma, Lingfan Yu, Yu Gai, Tianjun Xiao, Tong He, George Karypis, Jinyang Li, and Zheng Zhang. Deep graph library: A graph-centric, highly-performant package for graph neural networks. *arXiv preprint arXiv:1909.01315*, 2019.
- [19] JEDEC. Ddr sdram standard jesd79-4d, 2021.
- [20] Myoungsoo Jung. Hello bytes, bye blocks: PCIe storage meets compute express link for memory expansion (cxl-ssd). In *Proceedings of the 14th ACM Workshop on Hot Topics in Storage and File Systems*, pages 45–51, 2022.
- [21] Shao-Peng Yang, Minjae Kim, Sanghyun Nam, Juhung Park, Jinyong Choi, Eyee Hyun Nam, Eunji Lee, Sungjin Lee, and Bryan S Kim. Overcoming the memory wall with {CXL-Enabled}{SSDs}. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*, pages 601–617, 2023.
- [22] Miryeong Kwon, Sangwon Lee, and Myoungsoo Jung. Cache in hand: Expander-driven cxl prefetcher for next generation cxl-ssd. In *Proceedings of the 15th ACM Workshop on Hot Topics in Storage and File Systems*, pages 24–30, 2023.
- [23] Minseon Ahn, Andrew Chang, Donghun Lee, Jongmin Gim, Jungmin Kim, Jaemin Jung, Oliver Rebbholz, Vincent Pham, Krishna Malladi, and Yang Seok Ki. Enabling cxl memory expansion for in-memory database management systems. In *Proceedings of the 18th International Workshop on Data Management on New Hardware*, pages 1–5, 2022.
- [24] Huaicheng Li, Daniel S Berger, Lisa Hsu, Daniel Ernst, Pantea Zardoshti, Stanko Novakovic, Monish Shah, Samir Rajadnya, Scott Lee, Ishwar Agarwal, et al. Pond: Cxl-based memory pooling systems for cloud platforms. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, pages 574–587, 2023.
- [25] Kyungsan Kim, Hyunseok Kim, Jinin So, Wonjae Lee, Junhyuk Im, Sungjoo Park, Jeonghyeon Cho, and Hoyoung Song. Smt: Software-defined memory tiering for heterogeneous computing systems with cxl memory expander. *IEEE Micro*, 43(2):20–29, 2023.
- [26] Hasan Al Maruf, Hao Wang, Abhishek Dhanotia, Johannes Weiner, Niket Agarwal, Pallab Bhattacharya, Chris Petersen, Mosharaf Chowdhury, Shobhit Kanaujia, and Prakash Chauhan. Tpp: Transparent page placement for cxl-enabled tiered-memory. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, pages 742–755, 2023.
- [27] Yan Sun, Yifan Yuan, Zeduo Yu, Reese Kuper, Chihun Song, Jinghan Huang, Houxiang Ji, Siddharth Agarwal, Jiaqi Lou, Ipoom Jeong, et al. Demystifying cxl memory with genuine cxl-ready systems and devices. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, pages 105–121, 2023.
- [28] Donghyun Gouk, Miryeong Kwon, Hanyeoreum Bae, Sangwon Lee, and Myoungsoo Jung. Memory pooling with cxl. *IEEE Micro*, 43(2):48–57, 2023.
- [29] Blaise Tine, Krishna Praveen Yalamarthy, Fares Elsabbagh, and Kim Hyesoon. Vortex: Extending the risc-v isa for gpgpu and 3d-graphics.

- In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, pages 754–766, 2021.
- [30] Animesh Jain, Amar Phanishayee, Jason Mars, Lingjia Tang, and Genady Pekhimenko. Gist: Efficient data encoding for deep neural network training. In *2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*, pages 776–789. IEEE, 2018.
 - [31] Yanjie Gao, Yu Liu, Hongyu Zhang, Zhengxian Li, Yonghao Zhu, Haoxiang Lin, and Mao Yang. Estimating gpu memory consumption of deep learning models. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pages 1342–1352, 2020.
 - [32] Xuan Peng, Xuanhua Shi, Hulin Dai, Hai Jin, Weiliang Ma, Qian Xiong, Fan Yang, and Xuehai Qian. Capuchin: Tensor-based gpu memory management for deep learning. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 891–905, 2020.
 - [33] NVIDIA. Nvidia h100 tensor core gpu architecture, 2022.
 - [34] Aaron Harlap, Deepak Narayanan, Amar Phanishayee, Vivek Se-shadri, Nikhil Devanur, Greg Ganger, and Phil Gibbons. Pipedream: Fast and efficient pipeline parallel dnn training. *arXiv preprint arXiv:1806.03377*, 2018.
 - [35] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Dehao Chen, Mia Chen, HyoukJoong Lee, Jiquan Ngiam, Quoc V Le, Yonghui Wu, et al. Gpipe: Efficient training of giant neural networks using pipeline parallelism. *Advances in neural information processing systems*, 32, 2019.
 - [36] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.
 - [37] Jie Ren, Samyam Rajbhandari, Reza Yazdani Aminabadi, Olatunji Ruwase, Shuangyan Yang, Minjia Zhang, Dong Li, and Yuxiong He. {Zero-offload}: Democratizing {billion-scale} model training. In *2021 USENIX Annual Technical Conference (USENIX ATC 21)*, pages 551–564, 2021.
 - [38] AMD. Rocr software 6.0.2.
 - [39] Intel. Intel oneapi, 2022.
 - [40] Compute Express Link Consortium. Compute express link specification revision 3.1, 2023.
 - [41] Samsung. Samsung develops industry’s first cxl dram supporting cxl 2.0, 2023.
 - [42] SK Hynix. Sk hynix develops ddr5 dram cxl memory to expand the cxl memory ecosystem.
 - [43] Micron. Micron cz120 memory expansion module.
 - [44] Intel. Phy interface for the pci express, sata, usb 3.2, displayport, and usb4 architectures, revision 6.2.1, 2023.
 - [45] Intel. Migration from direct-attached intel optane persistent memory to cxl-attached memory.
 - [46] Samsung. Cmm-h (cxl memory module, h: Hybrid).
 - [47] Shang Li, Zhiyuan Yang, Dhiraj Reddy, Ankur Srivastava, and Bruce Jacob. Dramsim3: A cycle-accurate, thermal-capable dram simulator. *IEEE Computer Architecture Letters*, 19(2):106–109, 2020.
 - [48] Pawan Harish, Vibhav Vineet, and PJ Narayanan. Large graph algorithms for massively multithreaded architectures. *International Institute of Information Technology Hyderabad, Tech. Rep. IIIT/TR/2009/74*, 2009.
 - [49] Anthony Danalis, Gabriel Marin, Collin McCurdy, Jeremy S Meredith, Philip C Roth, Kyle Spafford, Vinod Tipparaju, and Jeffrey S Vetter. The scalable heterogeneous computing (shoc) benchmark suite. In *Proceedings of the 3rd workshop on general-purpose computation on graphics processing units*, pages 63–74, 2010.
 - [50] Woosong Cheong, Chanho Yoon, Seonghoon Woo, Kyuwook Han, Daehyun Kim, Chulseung Lee, Youra Choi, Shine Kim, Dongku Kang, Geunyeong Yu, et al. A flash memory controller for 15 μ s ultra-low-latency ssd using high-speed 3d nand flash with 3 μ s read time. In *2018 IEEE International Solid-State Circuits Conference (ISSCC)*, pages 338–340. IEEE, 2018.